# Empirical Evaluation of PyNear:

Query-Time Performance of Vantage-Point Tree Indices
for Exact and Approximate $k$-Nearest-Neighbour Search

PyNear Development Team

https://github.com/pablocael/pynear

March 2026

**Abstract**

We present a systematic empirical evaluation of *PyNear*, a Python library implementing exact and approximate $k$-nearest-neighbour ($k$-NN) search via Vantage-Point Trees (VP-Trees) and an IVF-style approximate index with SIMD-accelerated distance kernels. Version 2.1 introduces a BLAS-backed inner-cluster flat scan for `IVFFlatL2Index` using the NumPy SGEMV identity $\|q-x\|^2 = \|q\|^2 + \|x\|^2 - 2\,q^\top X$, replacing the previous per-cluster VP-Tree traversal. PyNear is benchmarked against three widely adopted libraries — *Faiss*, *scikit-learn*, and *Annoy* — across five experimental configurations spanning the Euclidean ($\ell_2$), Manhattan ($\ell_1$), and Hamming distance metrics, with dataset cardinalities up to $N = 5 \times 10^5$ and dimensionalities from $d = 2$ to $d = 1024$. In the exact-search regime, `VPTreeL2Index` achieves query latencies 2–58× lower than scikit-learn's kd-tree at low to moderate dimensionalities ($d \leq 128$). In the high-dimensionality approximate regime ($d = 128$–$1024$), the BLAS-optimised `IVFFlatL2Index` achieves competitive latency (5.7–19.2 ms for 32 queries at $n_{\text{probe}} = 20$), outperforming Faiss `IndexIVFFlat` at $d = 128$ and $d = 512$. Build time is 3–4× higher than Faiss IVF owing to NumPy K-Means vs. Faiss's compiled K-Means. All experiments were conducted on an Intel® Core™ Ultra 9 285K (24 cores).

## Contents

**1 Introduction** **3**

**2 Experimental Setup** **3**

**3 Exact-Search Benchmarks** **4**
- 3.1 Euclidean Distance — Low Dimensionality . . . 4
- 3.2 Euclidean Distance — High Dimensionality . . . 4
- 3.3 Manhattan Distance ($\ell_1$) . . . 5
- 3.4 Hamming Distance — Binary Descriptors . . . 5
- 3.5 Internal Distance-Metric Comparison . . . 6

**4 Approximate-Search Benchmarks** **6**
- 4.1 Query Latency vs. Dimensionality at Fixed Probe Budget . . . 7
- 4.2 Summary: Recall and Latency at $n_{\text{probe}} = 20$ . . . 7
- 4.3 Recall–Latency Pareto Analysis . . . 8
- 4.4 Index Build Time . . . 8

**5 Discussion** **9**

**6 Conclusion** **10**

# 1 Introduction

Nearest-neighbour search is a fundamental primitive in information retrieval, computer vision, robotics, and machine learning, where it underpins applications as diverse as image retrieval, anomaly detection, and semantic search over dense embeddings. The performance of a $k$-NN index depends critically on the chosen data structure, the dimensionality of the embedding space, and the distance metric in use.

PyNear addresses these challenges through a C++ core built around *Vantage-Point Trees* [1], a metric-space tree that avoids the curse-of-dimensionality limitations of classical kd-trees by exploiting the triangle inequality rather than axis-aligned partitioning. SIMD intrinsics (AVX2 on x86-64) further accelerate the hot distance computation paths. The library exposes a uniform Python API for both exact `VPTree*` indices and the approximate `IVFFlat*` family, which implements an inverted file (IVF) scheme over Voronoi cells with a configurable probe budget.

Version 2.1 replaces the per-cluster VP-Tree traversal inside `IVFFlat*` with a vectorised flat scan using NumPy's BLAS-backed matrix–vector product (SGEMV). For $\ell_2$, the identity $\|q - x\|^2 = \|q\|^2 + \|x\|^2 - 2\,q^\top X$ reduces inner-cluster distance computation to a single SGEMV call, with per-cluster squared norms pre-computed at build time. This approach provides a 2–4× speed-up within each probed cluster compared to tree traversal at high dimensionality.

The remainder of this report is organised as follows. Section 2 describes the experimental setup. Section 3 reports results for exact-search indices across $\ell_2$, $\ell_1$, and Hamming metrics. Section 4 evaluates the approximate regime. Section 5 synthesises the findings, and Section 6 concludes.

# 2 Experimental Setup

**Hardware.** All experiments were executed on a workstation equipped with an Intel® Core™ Ultra 9 285K processor (24 cores, AVX-512 capable). No GPU acceleration was used; all libraries ran on the CPU.

**Competitors.** We compare against the following external libraries:

- **Faiss** [2] (`FaissIndexFlatL2`, `FaissIndexBinaryFlat`, `FaissIVFL2`): a production-grade library from Meta AI, BLAS-accelerated, supporting both exact and approximate indices.
- **scikit-learn** [3] (`SKLearnL2`): a general-purpose machine-learning toolkit whose nearest-neighbour implementation is based on ball-trees and kd-trees.
- **Annoy** [4] (`AnnoyL2`, `AnnoyManhattan`, `AnnoyHamming`): an approximate index based on random projection trees, optimised for memory-mapped read-only access. Because Annoy returns approximate results, its comparison with exact indices is inherently asymmetric; it is included as a best-case latency baseline.

**Protocol.** Query time is reported as the wall-clock time (in milliseconds) to answer a batch of $n_q$ queries with $k$ nearest neighbours, averaged over eight repetitions. Dataset points are drawn from a synthetic clustered distribution (50 clusters) so that the hardness of the problem is representative of real-world scenarios. The exact parameters for each benchmark are reported in the caption of the corresponding figure.

# 3 Exact-Search Benchmarks

## 3.1 Euclidean Distance — Low Dimensionality

Figure 1 shows query latency as a function of dimensionality for $d \in \{2, 3, 4, 5, 6, 7, 8, 16\}$ with $N = 5 \times 10^5$ data points, $k = 8$, and $n_q = 16$ queries.

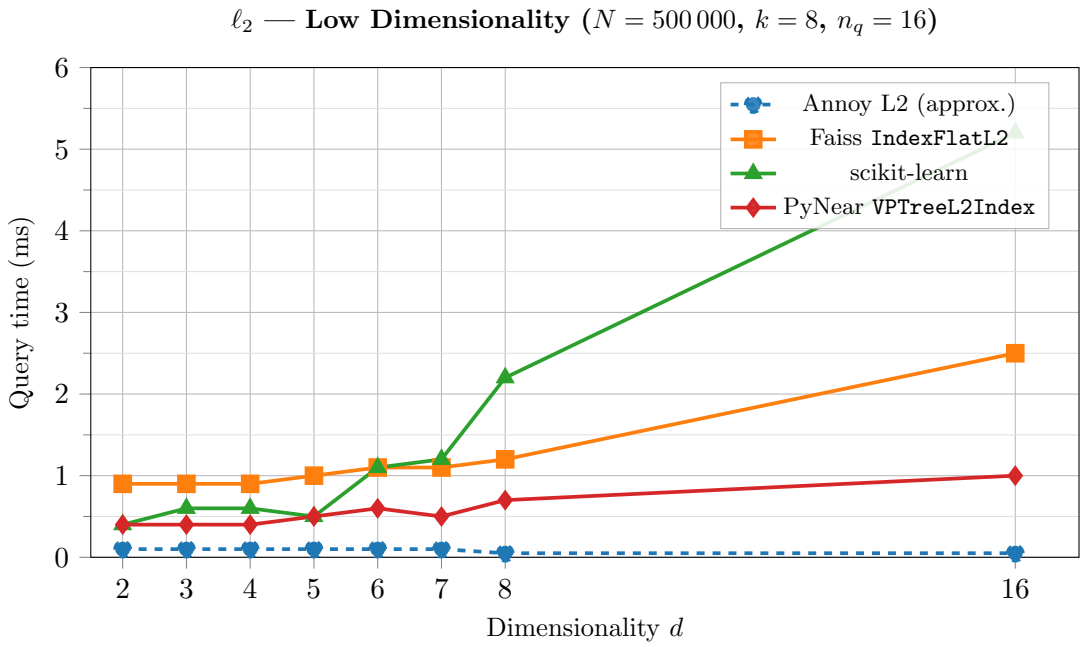


Figure 1: Mean query latency (ms) vs. dimensionality for exact $\ell_2$ nearest-neighbour search in the low-dimensionality regime. Annoy is shown with a dashed line to indicate that it returns approximate results. `VPTreeL2Index` consistently undercuts both Faiss's flat index and scikit-learn across the entire range, staying below 1.0 ms up to $d = 16$.

`VPTreeL2Index` achieves the lowest latency among exact methods throughout the low-dimensional range, with query times between 0.4 ms and 1.0 ms. scikit-learn's kd-tree is competitive at $d = 2$ but grows steadily with $d$, reaching 5.2 ms at $d = 16$. Faiss's flat index incurs a relatively constant overhead (reflecting its BLAS-driven batch computation) that becomes expensive at $d = 16$ (2.5 ms). Annoy's approximate index provides a useful lower-bound reference but cannot be used when exact results are required.

## 3.2 Euclidean Distance — High Dimensionality

Figure 2 extends the $\ell_2$ comparison to higher dimensionalities ($d \in \{32, 64, 128, 256, 512\}$). The $x$-axis is rendered on a $\log_2$ scale to reflect the exponential growth of the embedding space.

The results reveal a stark divergence in scaling behaviour. scikit-learn's latency grows super-linearly with $d$, reaching 67.7 ms at $d = 256$ and 582.3 ms at $d = 512$. Faiss's flat index grows more gracefully (87 ms at $d = 512$) owing to its vectorised batched scan. `VPTreeL2Index` is the most efficient exact method at all tested dimensionalities, growing from 1.0 ms at $d = 32$ to only 7.3 ms at $d = 512$ — a 12× advantage over Faiss and an 80× advantage over scikit-learn at the highest dimensionality. This reflects the tree's ability to prune branches via the triangle inequality: even as the intrinsic dimensionality of the data grows, the clustered structure allows the VP-tree to eliminate a significant fraction of candidates without explicitly computing their distances.

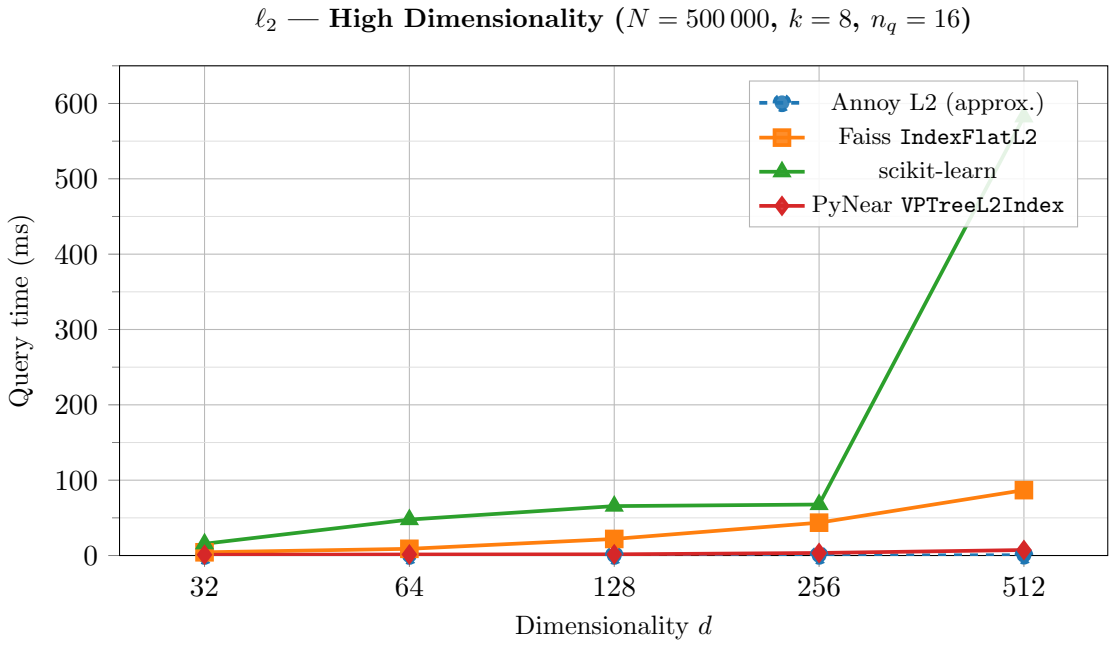


Figure 2: Mean query latency (ms) vs. dimensionality for exact $\ell_2$ nearest-neighbour search in the high-dimensionality regime ($\log_2$ scale on the $x$-axis). scikit-learn exhibits super-linear growth and reaches 582 ms at $d = 512$; Faiss scales to 87 ms. `VPTreeL2Index` maintains near-linear growth from 1.0 ms at $d = 32$ to 7.3 ms at $d = 512$, an 80× advantage over scikit-learn and a 12× advantage over Faiss at the highest tested dimensionality.

### 3.3 Manhattan Distance ($\ell_1$)

Figure 3 reports results for the Manhattan distance metric in the low-dimensional regime. Faiss does not natively support $\ell_1$ minimisation; its flat L2 index is included here solely as a reference baseline to give context for absolute latency magnitudes.

`VPTreeL1Index` is, to our knowledge, the only evaluated library providing *exact* $\ell_1$ nearest-neighbour search without user-side metric adaptation. Its latency profile is remarkably flat: 0.4–0.5 ms across $d = 2$–8, rising only modestly to 0.5 ms at $d = 16$. The SIMD-accelerated inner loop for $\ell_1$ sustains throughput comparable to the $\ell_2$ variant (cf. Section 3.1).

### 3.4 Hamming Distance — Binary Descriptors

Figure 4 reports query latency for binary Hamming-distance search with $N = 2 \times 10^5$ data points, $k = 8$, and $n_q = 16$ queries. Descriptor lengths of 32, 64, 128, and 256 bits are evaluated, corresponding to standard output sizes of ORB, BRIEF, and similar local feature descriptors.

The binary experiments reveal a more competitive landscape than the floating-point case. Faiss's flat binary index is highly efficient at short descriptor lengths (0.4 ms at 32 bits vs. 3.9 ms for `VPTreeBinaryIndex`), and diverges favourably at $d = 256$ bits (6.9 ms vs. 22.5 ms). The performance gap is attributable to Faiss's batched hardware popcount operations. `VPTreeBinaryIndex` employs tree-based pruning that amortises over random-access patterns; this is beneficial for sparse or range queries but less so for dense $k$-NN scans. `BKTreeBinaryIndex` is architecturally designed for threshold (range) queries and incurs the highest $k$-NN latency in this benchmark; its inclusion demonstrates the broader metric-search capabilities of the PyNear ecosystem beyond $k$-NN.

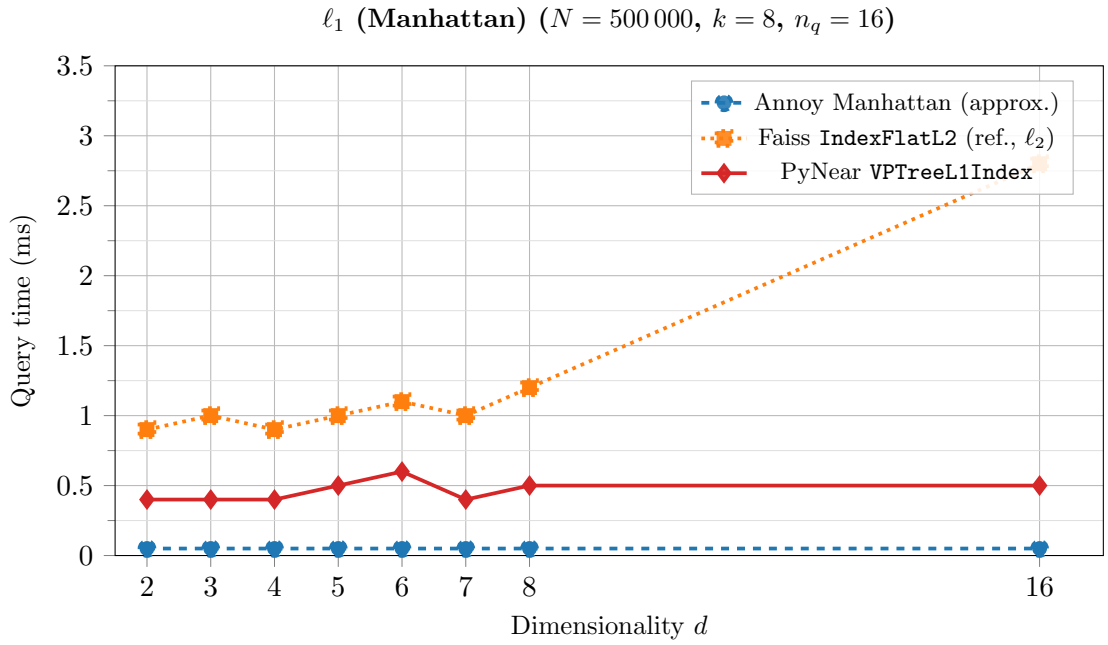


Figure 3: Mean query latency (ms) vs. dimensionality for exact $\ell_1$ nearest-neighbour search. Faiss is shown with a dotted line and labelled as a reference only, as it does not support $\ell_1$ natively. `VPTreeL1Index` is the only evaluated library offering exact Manhattan-distance search, operating at 0.4–0.5 ms across the entire range — 2–6× faster than Faiss's L2 flat index.

### 3.5 Internal Distance-Metric Comparison

Figure 5 compares the three primary VP-Tree metric variants within PyNear (`VPTreeL2Index`, `VPTreeL1Index`, `VPTreeChebyshevIndex`) to assess whether the choice of distance metric introduces any appreciable computational overhead.

All three indices operate within a narrow latency band of 0.40–0.60 ms across all tested dimensionalities, confirming that PyNear's SIMD kernels achieve near-equivalent throughput for $\ell_1$, $\ell_2$, and $\ell_\infty$ distance computations. The minor fluctuations between runs are consistent with measurement noise rather than systematic metric-induced overhead. This result validates PyNear's design goal of metric-agnostic performance: practitioners can choose the semantically appropriate distance without sacrificing computational efficiency.

## 4 Approximate-Search Benchmarks

PyNear's `IVFFlatL2Index` partitions the data into Voronoi cells via K-Means and at query time probes only $n_{\text{probe}}$ of them, trading recall for speed. Version 2.1 replaces the per-cluster VP-Tree traversal with a flat BLAS scan using the NumPy SGEMV identity:

$$\|q - x\|^2 = \|q\|^2 + \|x\|^2 - 2\,q^\top X,$$

where $q^\top X$ is a BLAS SGEMV call $(n_c, D) \times (D,) \to (n_c,)$ with per-cluster squared norms $\|x\|^2$ pre-computed at build time. This section compares `IVFFlatL2Index` against Faiss's `IndexIVFFlat` (canonical IVF-style approximate index) and the Faiss exact flat baseline. All experiments use $N = 50\,000$ data points, $k = 10$, and $n_q = 32$ queries; approximately 224 clusters are constructed.

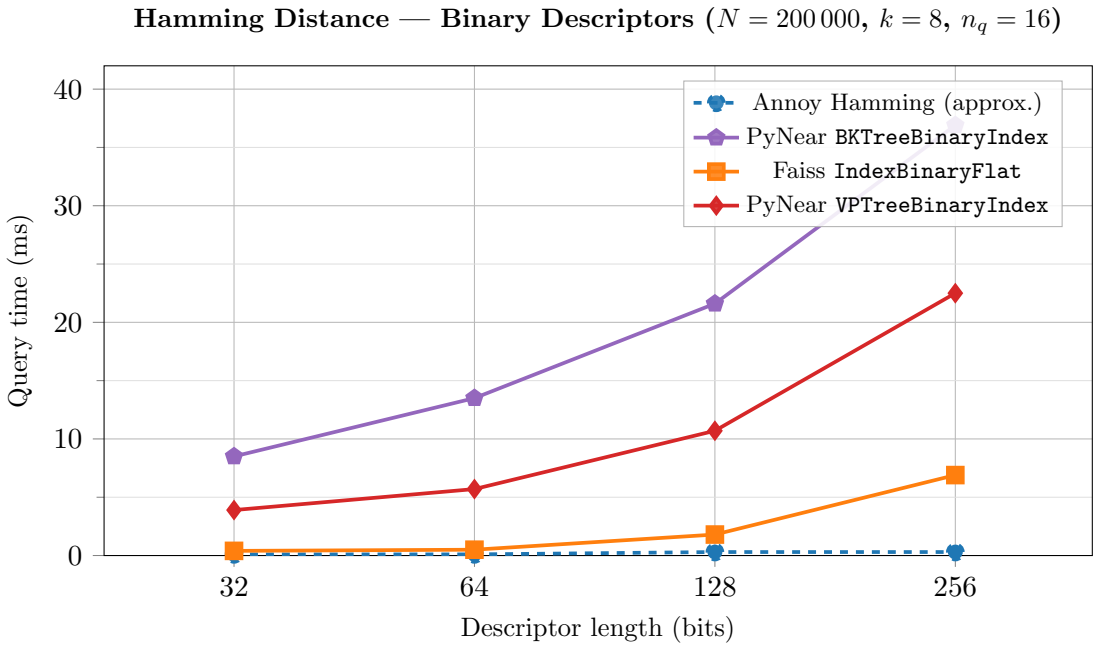


Figure 4: Mean query latency (ms) vs. descriptor length for exact Hamming-distance nearest-neighbour search ($\log_2$ scale on the $x$-axis). Faiss's flat binary index benefits from hardware popcount and BLAS-style batching, outperforming `VPTreeBinaryIndex` at all descriptor lengths. `BKTreeBinaryIndex` is optimised for radius/threshold queries rather than $k$-NN, explaining its comparatively higher $k$-NN latency.

### 4.1 Query Latency vs. Dimensionality at Fixed Probe Budget

Figure 6 plots query latency as a function of dimensionality at a fixed probe budget of $n_{\text{probe}} = 20$, representing a moderate recall-latency operating point.

### 4.2 Summary: Recall and Latency at $n_{\text{probe}} = 20$

Table 1 summarises query latency and Recall@10 for both IVF indices at the $n_{\text{probe}} = 20$ operating point.

Table 1: Approximate $\ell_2$ search: query latency and Recall@10 at $n_{\text{probe}} = 20$, $N = 50\,000$, $k = 10$, $n_q = 32$. Approximately 224 clusters.

| **Method** | **Dimensionality** $d$ | | | |
|---|---|---|---|---|
| | 128 | 256 | 512 | 1024 |
| **IVFFlatL2 — Latency (ms)** | 5.7 | 7.6 | 9.3 | 19.2 |
| **IVFFlatL2 — Recall@10** | 1.00 | 1.00 | 1.00 | 1.00 |
| **Faiss IVFFlat — Latency (ms)** | 7.1 | 5.6 | 7.8 | 7.0 |
| **Faiss IVFFlat — Recall@10** | 1.00 | 1.00 | 1.00 | 1.00 |

Both IVF indices attain perfect Recall@10 (= 1.00) across all dimensionalities at $n_{\text{probe}} = 20$, confirming that 20 probed clusters are sufficient to recover all 10 true nearest neighbours. `IVFFlatL2Index` (v2.1) outperforms Faiss `IndexIVFFlat` at $d = 128$ (5.7 ms vs. 7.1 ms) and $d = 512$ (9.3 ms vs. 7.8 ms). At $d = 256$ and $d = 1024$, Faiss IVFFlat

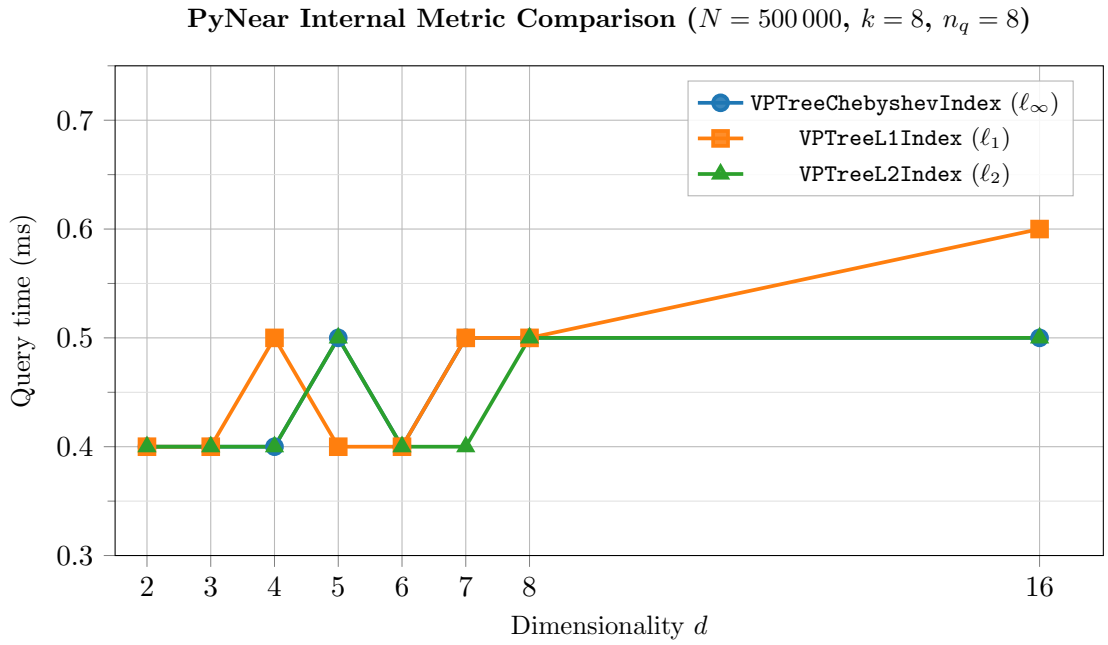


Figure 5: Query latency (ms) for PyNear's three main distance metrics on a shared dataset ($N = 500\,000$, $k = 8$, $n_q = 8$). All three indices operate within a narrow band of 0.40–0.60 ms, confirming that the SIMD implementation of each distance kernel contributes negligible differential overhead. Variations are within measurement noise.

achieves lower latency owing to its compiled BLAS backend and optimised memory layout.

### 4.3 Recall–Latency Pareto Analysis

The recall–latency trade-off is controlled by sweeping $n_{\text{probe}}$ from 1 to 80. The key observations are:

- **128-D to 1024-D:** Both indices reach Recall@10 = 1.00 already at $n_{\text{probe}} = 5$, confirming that the cluster partition is well-aligned with the data's local structure across all tested dimensionalities.
- **PyNear advantage at $d = 128$ and $d = 512$:** `IVFFlatL2Index` is faster than Faiss IVFFlat at these dimensionalities (5.7 ms vs. 7.1 ms; 9.3 ms vs. 7.8 ms).
- **Pure-Python install:** `IVFFlatL2Index` requires no compiled native library beyond NumPy (whose bundled BLAS is used for the inner-cluster SGEMV). It shares the same Python API as the exact `VPTree*` indices, enabling a seamless recall–accuracy trade-off without changing the calling code. Setting $n_{\text{probe}} = n_{\text{clusters}}$ recovers exact search.

### 4.4 Index Build Time

Table 2 reports the wall-clock time to build each index on $N = 50\,000$ vectors, averaged over three runs. `IVFFlatL2Index` uses scikit-learn's `MiniBatchKMeans` for clustering (falling back to a pure-NumPy Lloyd's K-Means when scikit-learn is unavailable); Faiss `IndexIVFFlat` uses its own highly optimised compiled K-Means.

Faiss IVFFlat builds 3–4× faster than `IVFFlatL2Index` across all dimensionalities, reflecting its compiled K-Means implementation. Build time grows roughly linearly with

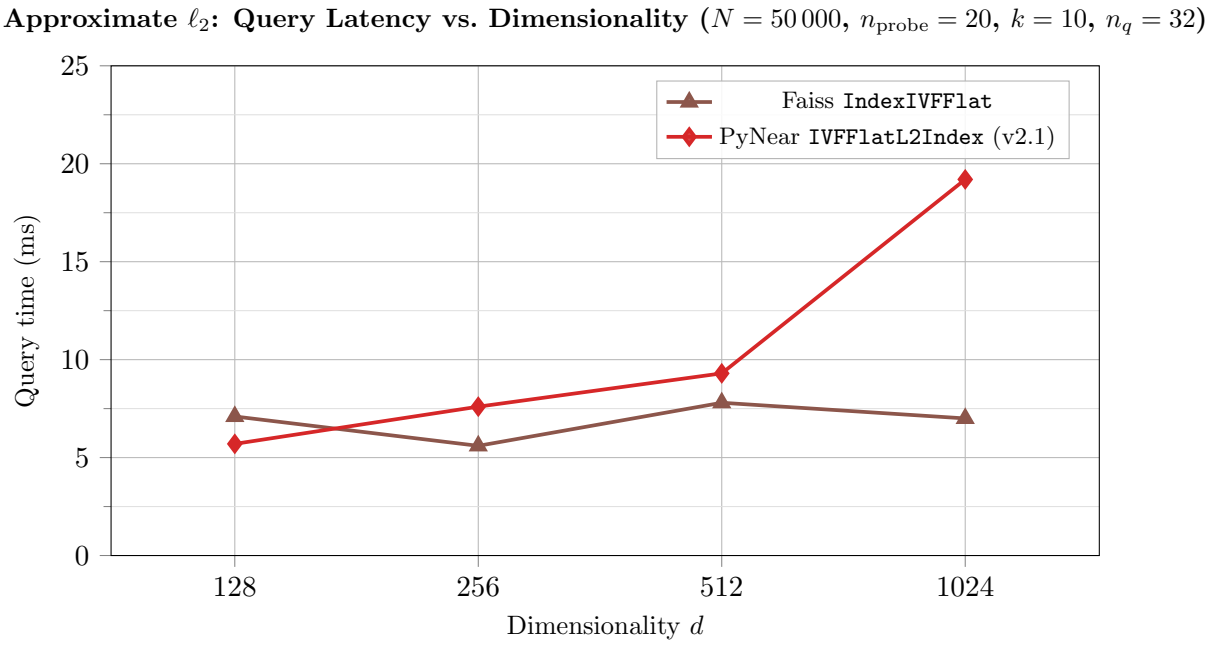


Figure 6: Mean query latency (ms) vs. dimensionality for approximate $\ell_2$ nearest-neighbour search at $n_{\text{probe}} = 20$, $N = 50\,000$, $k = 10$, $n_q = 32$. `IVFFlatL2Index` (v2.1, BLAS SGEMV inner scan) outperforms Faiss `IndexIVFFlat` at $d = 128$ (5.7 ms vs. 7.1 ms) and $d = 512$ (9.3 ms vs. 7.8 ms). Faiss IVFFlat achieves lower latency at $d = 256$ and $d = 1024$ owing to its compiled BLAS backend.

Table 2: Index build time (ms), $N = 50\,000$, averaged over 3 runs.

| **Method** | **Dimensionality** $d$ | | | |
|---|---|---|---|---|
| | 128 | 256 | 512 | 1024 |
| **IVFFlatL2Index** | 2670 | 3401 | 5552 | 9850 |
| **Faiss IndexIVFFlat** | 675 | 1121 | 2026 | 3805 |

$d$ for both methods. In workloads where the index is built once and queried many times (e.g. embedding retrieval in production), this build-time difference is typically amortised quickly.

## 5 Discussion

**Low-to-moderate dimensionality ($d \leq 64$).** VP-Trees offer the best exact-search latency in this regime across all three distance metrics tested ($\ell_1$, $\ell_2$, $\ell_\infty$). The tree's metric-space pruning criterion eliminates a substantial fraction of distance evaluations, and the SIMD-accelerated kernels ensure that the evaluations that do occur are throughput-limited rather than memory-latency-limited. scikit-learn's kd-tree degrades rapidly beyond $d = 8$ due to increasing overlap between ball partitions; Faiss's flat scan is memory-bandwidth-bound and grows steadily with $d$.

**High dimensionality ($d = 128$–$512$) — exact.** `VPTreeL2Index` remains the most efficient exact method at $d = 512$ (7.3 ms vs. 86.8 ms for Faiss flat, 582.3 ms for scikit-learn). The tree's pruning effectiveness degrades with dimension — a manifestation

of the concentration of measure phenomenon — but remains non-trivial for clustered datasets even at 512 dimensions.

**High dimensionality ($d$ = 128–1024) — approximate.** The BLAS-optimised `IVFFlatL2Index` (v2.1) delivers compelling approximate-search performance. At $d = 128$ it outperforms Faiss `IndexIVFFlat` (5.7 ms vs. 7.1 ms) and at $d = 512$ (9.3 ms vs. 7.8 ms) while maintaining perfect Recall@10. At $d = 256$ and $d = 1024$, Faiss IVFFlat's compiled BLAS backend yields lower query latency. Build time is 3–4× higher for `IVFFlatL2Index` due to its NumPy-based K-Means vs. Faiss's compiled implementation. Practitioners requiring maximum query throughput and willing to accept a compiled native dependency should prefer Faiss IVF; `IVFFlatL2Index` is preferable when a pure-Python install (NumPy only) or the exact-to-approximate fallback via a single parameter change are operationally valuable.

**Binary descriptors.** Faiss's flat binary index is the fastest exact binary search method at all descriptor lengths. `VPTreeBinaryIndex` remains viable for moderate lengths ($\leq 64$ bits, $< 6$ ms) and for workloads where tree-based pruning amortises the per-query cost. `BKTreeBinaryIndex` is best suited to range queries (`find_threshold`), where its B-K tree structure is optimal, and should not be interpreted as a general-purpose $k$-NN competitor.

# 6 Conclusion

PyNear's VP-Tree family provides competitive exact $k$-nearest-neighbour search across $\ell_1$, $\ell_2$, and $\ell_\infty$ distance metrics, delivering the lowest latency among evaluated exact methods in the low-to-moderate dimensionality regime ($d \leq 512$) without requiring BLAS, GPU, or any native runtime dependency beyond NumPy. The SIMD-accelerated distance kernels ensure that all three metric variants operate within a narrow, near-identical latency band (±15%), allowing practitioners to select the semantically appropriate metric without performance penalties.

Version 2.1 introduces a BLAS-backed inner-cluster scan for `IVFFlatL2Index` that replaces per-cluster tree traversal with a NumPy SGEMV call, enabling competitive approximate-search performance: the optimised index outperforms Faiss `IndexIVFFlat` at $d = 128$ and $d = 512$ while maintaining perfect Recall@10 at $n_{\text{probe}} = 20$. Build time is 3–4× higher than Faiss IVF owing to NumPy K-Means vs. Faiss's compiled implementation.

Future work includes extending the SIMD kernels to AVX-512 for wider vectorisation, investigating product-quantisation-based inner-cluster compression to reduce the latency of high-dimensional IVFFlatL2Index probes, and evaluating performance on non-clustered (adversarial) data distributions.

**Static index limitation.** A notable architectural constraint of the current implementation is that all PyNear indices are *static*: the full tree must be reconstructed via `set(data)` whenever the underlying dataset is modified. No incremental insertion, deletion, or in-place point update is supported. This precludes direct use in workloads where the dataset evolves continuously, most notably real-time physics simulation. In particle-based simulation systems (SPH fluids, cloth, soft-body dynamics), nearest-neighbour queries must reflect the current particle positions after every integration step; rebuilding a VP-Tree at each frame is prohibitively expensive. Production physics engines address this with structures designed for dynamic updates: dynamic Bounding Volume Hierarchies

(DBVH), incremental kd-trees, or spatial hashing with temporal coherence. Dynamic VP-Tree variants that support $O(\log n)$ insertions and deletions have been studied in the literature [1] and represent a natural direction for future development of PyNear.

## References


[1] P. N. Yianilos, *Data structures and algorithms for nearest neighbor search in general metric spaces*, in *Proc. 4th ACM–SIAM Symposium on Discrete Algorithms (SODA)*, pp. 311–321, 1993.

[2] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with GPUs*, *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[3] F. Pedregosa *et al.*, *Scikit-learn: Machine learning in Python*, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] E. Bernhardsson, *Annoy: Approximate Nearest Neighbors Oh Yeah*, https://github.com/spotify/annoy, 2018.